



An Analysis of Training Models Utilizing OpenAI: Testing the Abilities of RAG vs. Fine-tuning

Annalia Schoenherr, BS Computer Science Candidate, CoAS; Igri Fishta, BS Computer Science Candidate, CoAS; CJ Chung, PhD, Professor, CoAS

College of Arts and Sciences, Lawrence Technological University



Abstract

This project compares various OpenAI large language models to assess their ability to answer user-defined questions about Lawrence Tech's undergraduate computer science courses. Four models were evaluated: the free ChatGPT interface (model 4o-mini), the premium ChatGPT interface (model o3-mini-high), a fine-tuned model (base model gpt-4o-mini-2024), and a Retrieval Augmented Generation (RAG) model (model 4o-mini). Each model was provided with course data—name, description, and prerequisites—using different methods: direct interface use, JSONL training, or embeddings. Fifteen questions of varying difficulty were posed to each model, with responses scored for relevance, clarity/conciseness, and accuracy. The evaluation highlights that each model has unique benefits and trade-offs, such as cost, integration ease, and customization, making model choice dependent on specific application requirements and resource constraints. Results showed that the premium ChatGPT model scored highest with a score of 4.93 out of 5, while the RAG model accumulated a score of 4.04 out of 5. Even with a higher score, the premium ChatGPT model was not an effective way to implement question-answering tasks about MCS courses due to limited user accessibility. Further improvements on the datasets for the RAG and fine-tuning models can be made and are more applicable for a future development of a functional chatbot on the MCS website.

Introduction

Building effective chatbots to assist with specialized tasks requires not only advanced language models but also substantial, high-quality training data. Our team previously attempted to fine-tune OpenAI's GPT-4.0-mini model on a different subject, which showed some promise but ultimately proved to be inaccurate and unreliable [4]. The primary issue stemmed from an insufficient amount of domain-specific training data, which limited the model's ability to generate precise and relevant responses. Generating synthetic data to address this shortfall required considerable time, effort, and resources, making it an impractical solution.

This experience highlighted the need for a more efficient and scalable approach, which led us to explore the use of Retrieval-Augmented Generation (RAG) models. Unlike traditional fine-tuning methods, RAG combines the strengths of pre-trained language models with the ability to retrieve specific, relevant data from external knowledge bases in real-time [3]. This hybrid approach eliminates the need for extensive fine-tuning while improving the accuracy and reliability of responses, especially in scenarios where training data is limited.

To achieve the output from a RAG model, data must be collected in the form of structured or unstructured text data. In the scope of this project, pdf files were used. First, when a prompt is provided, a search is performed on textual data stored in the database utilized by the model. An algorithm is used to obtain and rank information based on relevance to the prompt provided. This is known as the retrieval step. Next comes augmentation, where the selected relevant documents are combined with the original prompt/query from the user in order to provide context for the language model to answer the question. The generation step in RAG involves using a generative model, usually an existing large language model, to process the context from the retrieval step and the user's prompt to create and format a response [5][6]. This response is then returned to the user and concludes an iteration of using RAG for a question-answering task.

Our project focuses on analyzing the effectiveness of RAG models against other OpenAI models to answer questions about computer science courses at Lawrence Tech. to help students understand course scheduling opportunities. This problem is particularly significant, as a well-designed chatbot could streamline academic planning, reduce the burden on advisors, and empower students to make informed decisions about their education. By addressing the challenges of training data limitations and model reliability, this project aims to demonstrate the potential of RAG in building domain-specific, user-friendly chatbots for educational purposes.

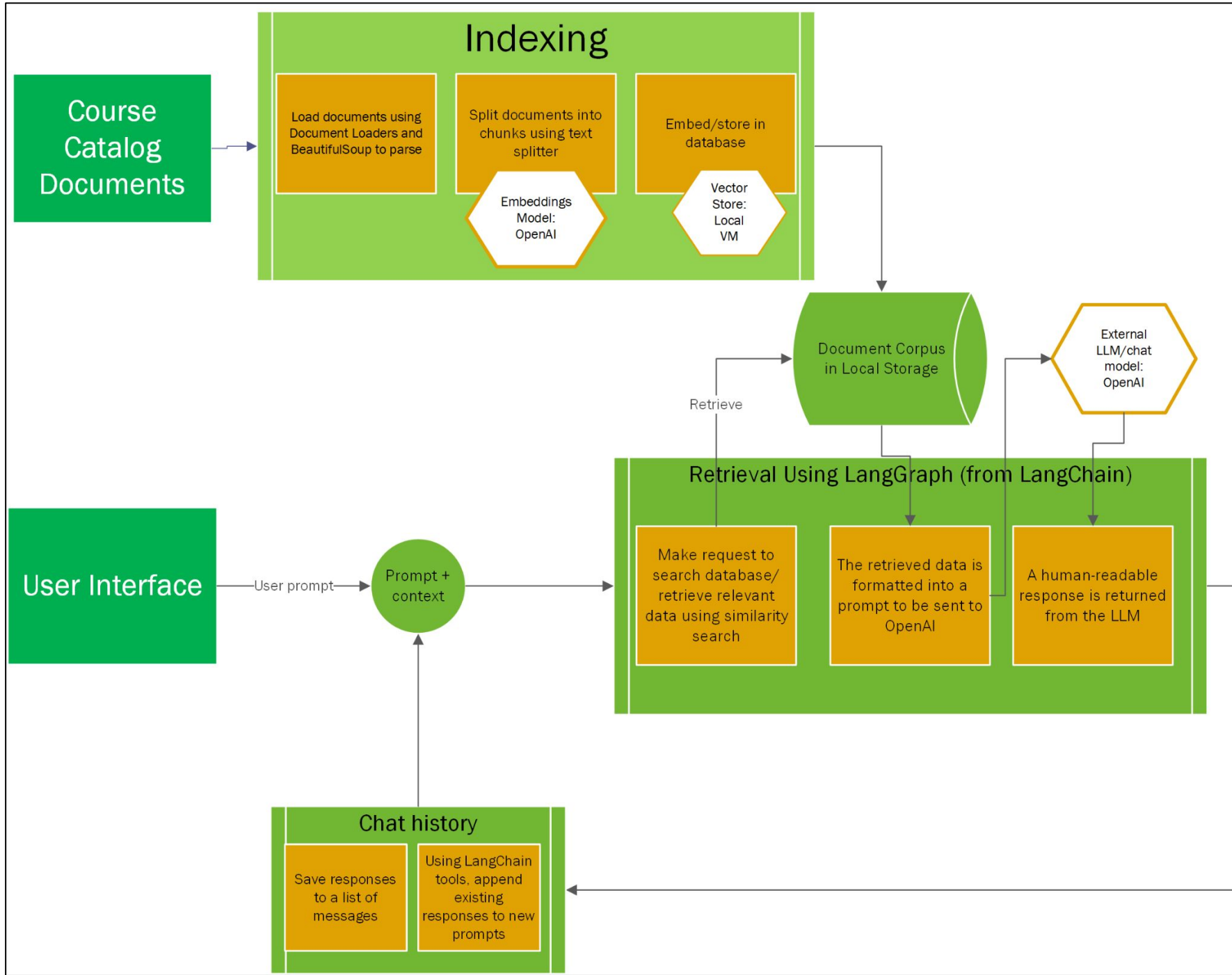
Design

Design For Retrieval-Augmented Generation (RAG) Model

To develop our RAG model, we used a LangChain and LangGraph design. It can be defined as follows:

- Corpus creation: Documents are collected from the LTU course catalog using a web scraping tool and are stored as pdf files.
- Indexing:
 - Load: Import the relevant documents (in raw text format) by utilizing LangChain's Document Loaders to convert the files into embeddings [5].
 - Split: Use LangChain's text splitter to divide large documents into manageable chunks and embed them/convert them into numerical format.
 - Store: The "chunks" are then indexed and stored in a local database for future retrieval. [6]
- Retrieval/Generation:
 - The user enters a prompt through the user interface
 - The prompt is formatted with context including instructions for how the LLM should respond and (if applicable) conversation history from previous queries.
 - A function is used to embed and send the prompt to the document corpus.
 - Documents with similarities to the prompt are returned from the database.
 - The returned documents are added to a new prompt that is sent to OpenAI for processing [2].
 - OpenAI provides a response in human-readable format
 - The returned docs and OpenAI response are stored in a chat history variable
 - The OpenAI response is returned to the user

- Tools/frameworks used:
- LangChain and LangGraph (application structure and workflow)
 - OpenAI API is used for Natural Language Processing tasks
 - JavaScript application to be used for user interface



Data Collection

The dataset for the RAG model consists of multiple text files containing information about computer science courses at Lawrence Tech [7], and their sequences. Data collection included the development of a web-scraping Python script that obtains each course name, course description, course prerequisites, and other details from their course description web pages and saves them to an external text file. An example of the converted output can be seen as follows:

The dataset for the fine-tuned model required a JSONL formatted training file containing predefined prompts and responses for the model to use for training. This data was created using ChatGPT by providing the information about the undergrad courses and then using the prompt seen

Test Questions

To quantitatively analyze the quality of the responses from the RAG model, a set of questions were developed and asked to the four question-answering models: OpenAI's ChatGPT interface, ChatGPT Plus, an OpenAI 4.0 model fine-tuned with the same data, and our RAG model. Each question ranges in difficulty to test real questions that may be asked in the real world. Easy questions, highlighted in green, ask simple, straightforward questions about the data. Medium level questions, highlighted in yellow, ask more complex questions that are not explicitly stated in the data. Difficult questions, highlighted in red, have multiple parts or require information that the model's haven't been trained on yet. The questions used for this project can be seen in the table right:

1	What is the prerequisite for MCS 3513 Software Architecture?
2	List 7 important topics that MCS 2514 (Computer Science 2) covers.
3	What class should I take after MCS 3643 Software Engineering I?
4	Can I take Computer Science 1 and Computer Science 2 in the same semester?
5	How many credit hours is Computer Science 2?
6	Can a student take MCS 3123 without MCS 2124?
7	List 10 MCS courses that cover the most valuable topics for a good understanding of undergraduate Computer Science.
8	If I am really good and advanced at CS, can I take upper-level courses in my sophomore year?
9	Can you recommend any courses that focus specifically on artificial intelligence and machine learning?
10	A student can take only two classes due to financial problems. He is currently taking MCS 3643. What course do you recommend him to take next semester?
11	Can I take Senior Project 1 if I am not a senior student?
12	What options do I have for independent study or research projects within the computer science department?
13	Can you suggest any courses that offer a good introduction to programming for someone with minimal background?
14	A student is good at math and he tries to take MCS2543 without MCS 1074 Precalculus. Is this possible? How would you handle this?
15	List all the classes (number and name) that require MCS 2534 Data Structures as a prerequisite.

Question Analysis

The responses obtained from each of the prompts were scored on relevance, clarity, and accuracy. The following table depicts each scoring category:

Grading Scale:	0	1	2	3	4	5
Relevance	Completely irrelevant to the question asked	Barely relevant, almost entirely off topic	Mostly irrelevant, sparse relevant content	Partially relevant	Mostly relevant	Directly answers the question with pertinent details
Clarity/conciseness	Completely incomprehensible and overly lengthy	Very unclear, hard to understand with minimal focus on desired content	Poorly explained and difficult to follow and includes unnecessary details	Understandable but somewhat unclear in parts. Includes 3 sentences or more	Mostly clear, minor ambiguities and slightly verbose	Exceptionally clear and easy to understand, delivers necessary information without unnecessary details
Accuracy	Entirely inaccurate/misleading	Highly inaccurate with very little correct information	Contains more inaccuracies than accurate facts	Contains more inaccuracies than accurate facts	Information is accurate with one or two minor	All information is accurate and verifiable

The responses from each model scored relatively high. The full depiction of the mean score for each response can be seen in the bar graph at the bottom of the section. An example of a low scoring question can be seen in the results of the following question:

Question: "What is the prerequisite for MCS 3513 Software Architecture?"

Fine-tuned model response: "Requires an undergraduate-level background with successful completion of MCS 2513 and MCS 3233 (with a minimum grade of D-)."

This response received a score of 4 for relevance, as it does attempt to answer the question. It also received a clarity/conciseness score of 4, as it does not include unnecessary information. The accuracy score determined for this response was 0, as MCS 2513 and MCS 3233 are not real courses offered during the 2025 Fall semester at Lawrence Tech.

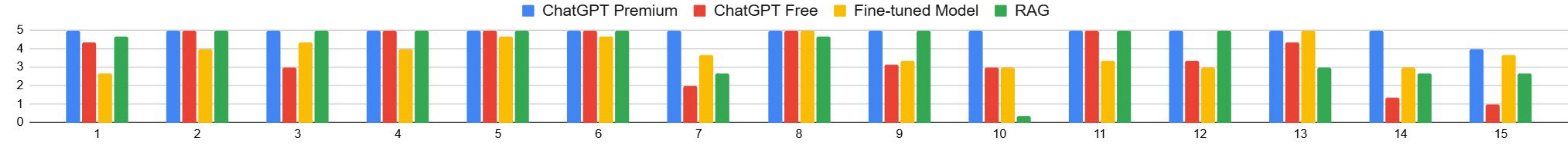
An example of a perfect score can be seen in ChatGPT Premium's response to the same question:

Question: "What is the prerequisite for MCS 3513 Software Architecture?"

ChatGPT Premium Response: "MCS 3513 requires completion of both MCS 2513 and MCS 2514 with a minimum grade of D-."

In this response, the courses listed are all real courses and the correct answer to the question provided. The response was also relevant and concise, showing that the OpenAI model is fully qualified to comprehend and respond to information provided to it.

Overall Score per Question



Conclusion/Takeaways

The results of the project have ultimately concluded that all models have a strong ability to comprehend textual information provided to it and provide somewhat accurate responses based on this information. The breakdown of each model's capabilities are seen as follows:

ChatGPT (Free Version):

While the free version of ChatGPT was able to correctly comprehend the information it was provided and return accurate responses, it is not feasible for use in a chatbot setting as it does not allow stored input or access from external application. For the development of an application that aids students in understanding computer science course offerings and requirements, a large set of data is required. The free ChatGPT interface requires users to input all information at the beginning of their session before asking questions. Users are not expected to know any of the course information, so this would not be feasible for them to implement.

ChatGPT (Premium Version):

The premium version of ChatGPT scored higher than the free version of ChatGPT in all categories of accuracy, clarity, and relevance. This is due to the usage of a more equipped model. While this model proved to be able to process and respond to text more accurately than the other models, it is also not feasible for the implementation of a chatbot, as it runs into the same issue of lacking the ability to store data or be accessed from an external application. Once again, users would have to input all of the course data before making queries, which is not something they are expected to do in the scenario of a question-answering chatbot for academic advising. In addition to the lack of storing data, the premium model incurs a fee for usage, which is also a drawback for users looking to implement this model to solve the problem of understanding the academic offerings for their major.

Fine-tuned OpenAI Model:

The fine-tuned model solves the problem of storing data, as this method allows for training on a reusable set of data. The drawback of this is the format and quantity of the data. In order to have the most accurate results from the dataset, a large set of JSONL data is required for training. Creating this dataset has proven to be an inefficient and expensive task, as it requires a significant amount of manual processing. In addition to the creation of files utilizing text generation models like OpenAI. While the data differed slightly both quantity and quality wise, the fine-tuned OpenAI model had high scores the least often in both clarity and accuracy, which are both essential for users when it comes to output provided by a chatbot.

RAG Model:

Use of RAG for question-answering also solves the problem of data storage, as the method allows for storage of a set of text files or pdfs that can be reused for each query. The model had the second highest score for overall responses of 25.8%. This score is believed to be lower than ChatGPT Premium due to the data it uses for each query. In ChatGPT Premium, all of the academic data is referenced, while the RAG model uses only chunks of the overall data from the imported pdf files with corresponding information. It is believed that scores for this model can be improved with the utilization of more relevant data files.

What Was Limited:

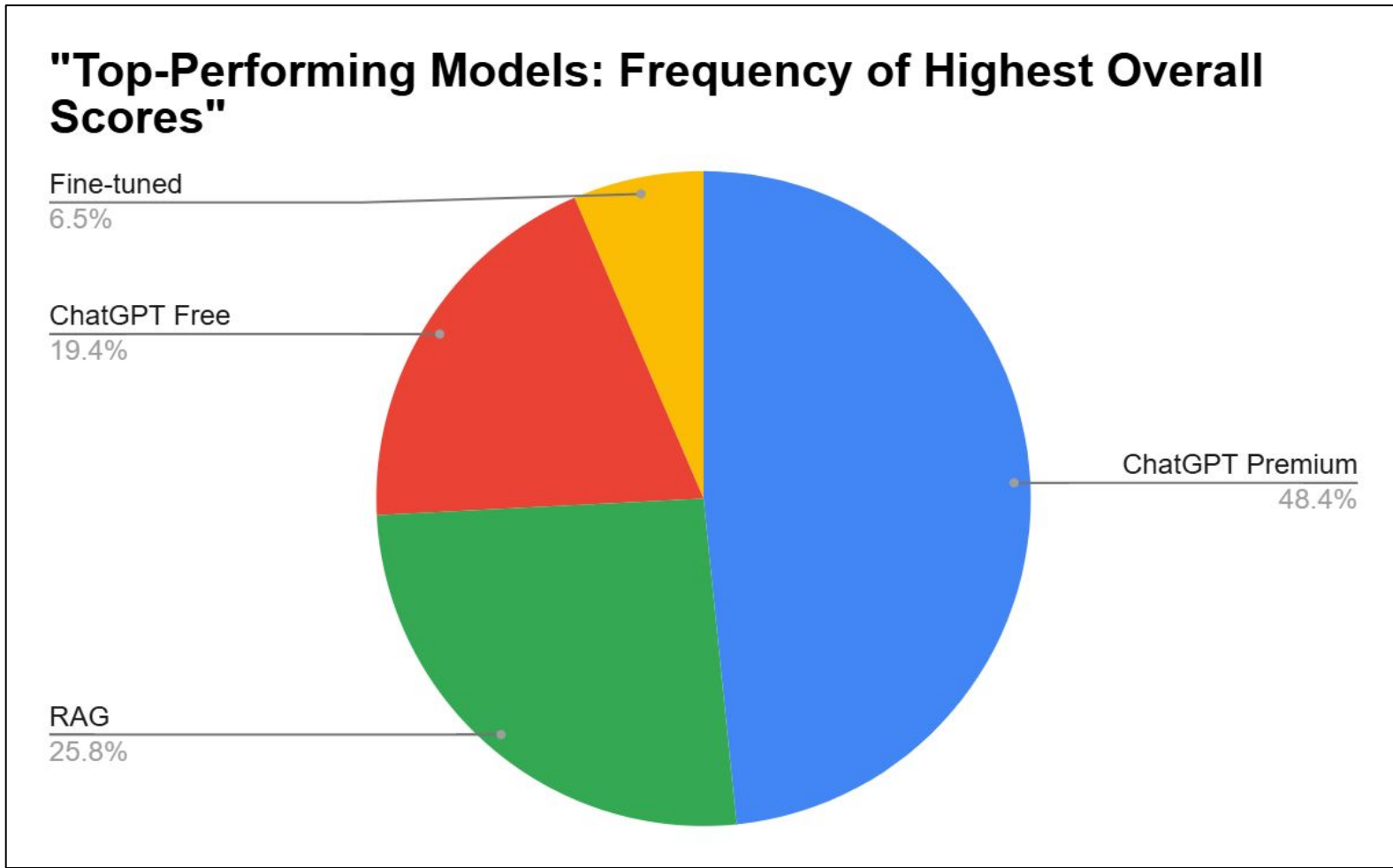
Limitations of testing these models included the restriction of model usage by OpenAI, as its free tiers limit the amount of queries made by a user to an average of 8-10 questions based on token usage. This not only made testing difficult, but solidified the understanding that this model would not be feasible for a question-answering application, as the application would quickly limit questions asked by the users.

Another limitation included the cost of using each model. While relatively low, the cost of training and using each model could result in a significant expense for maintaining the chatbot application.

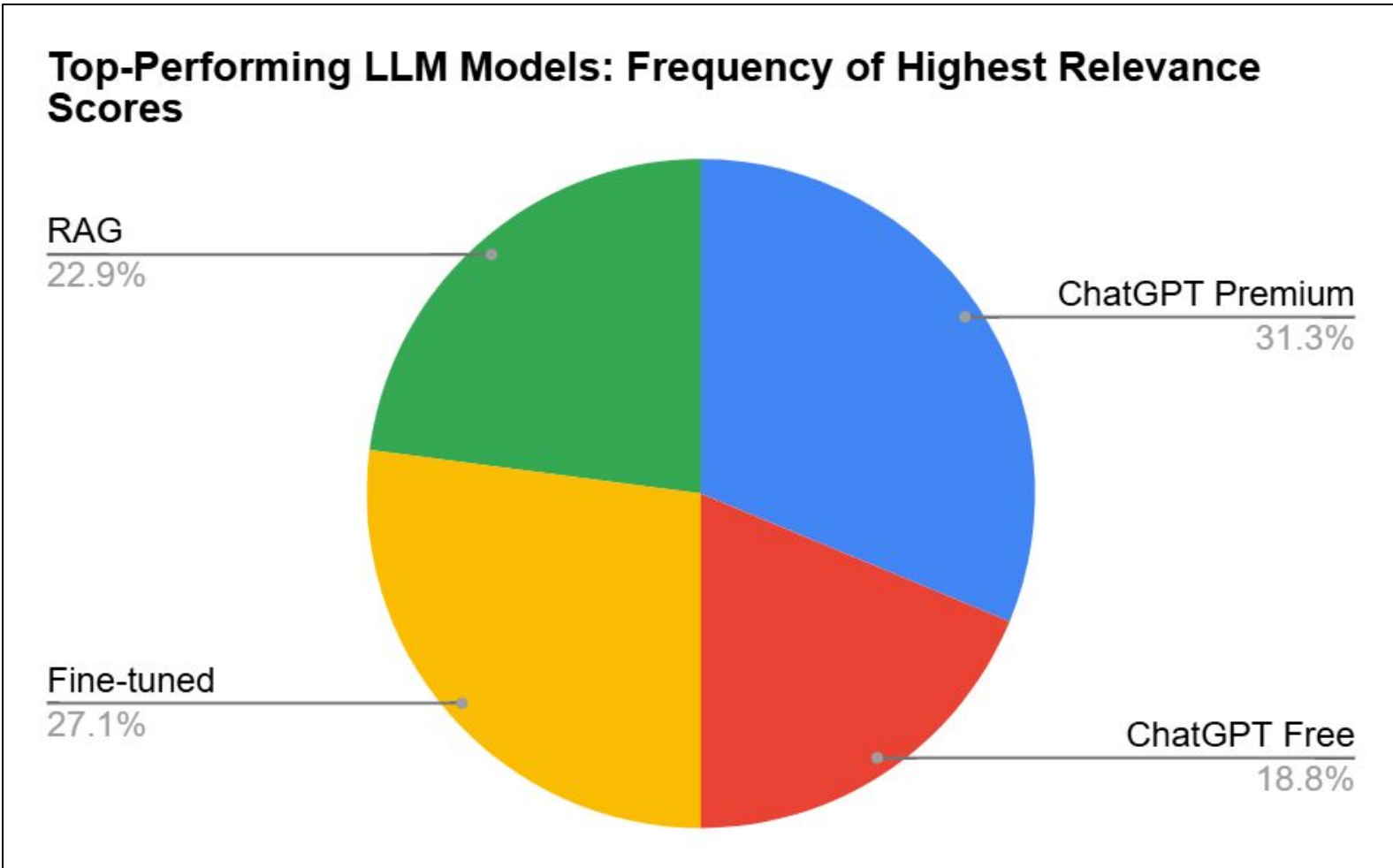
Data acquisition was also a challenge in this project, as each model requires a different format of data. The ChatGPT models required an explanation of data through the user interface, the fine-tuned model requires a JSONL file with prompts and responses, and the RAG model requires pdf files of the data. This made it difficult to confidently compare the success of each model's responses, as each was using slightly different datasets. Further refinement of data for future models is worth exploring to truly understand the effectiveness of each model.

The advancement of OpenAI's models and interfaces also evolved during the course of this project. This presented the challenge of maintaining consistency while testing each model, as the natural capabilities of OpenAI were evolving faster than the completion of testing. New abilities included new model development, advanced data storage/conversation history, updated user interfaces and the depreciation of interfaces.

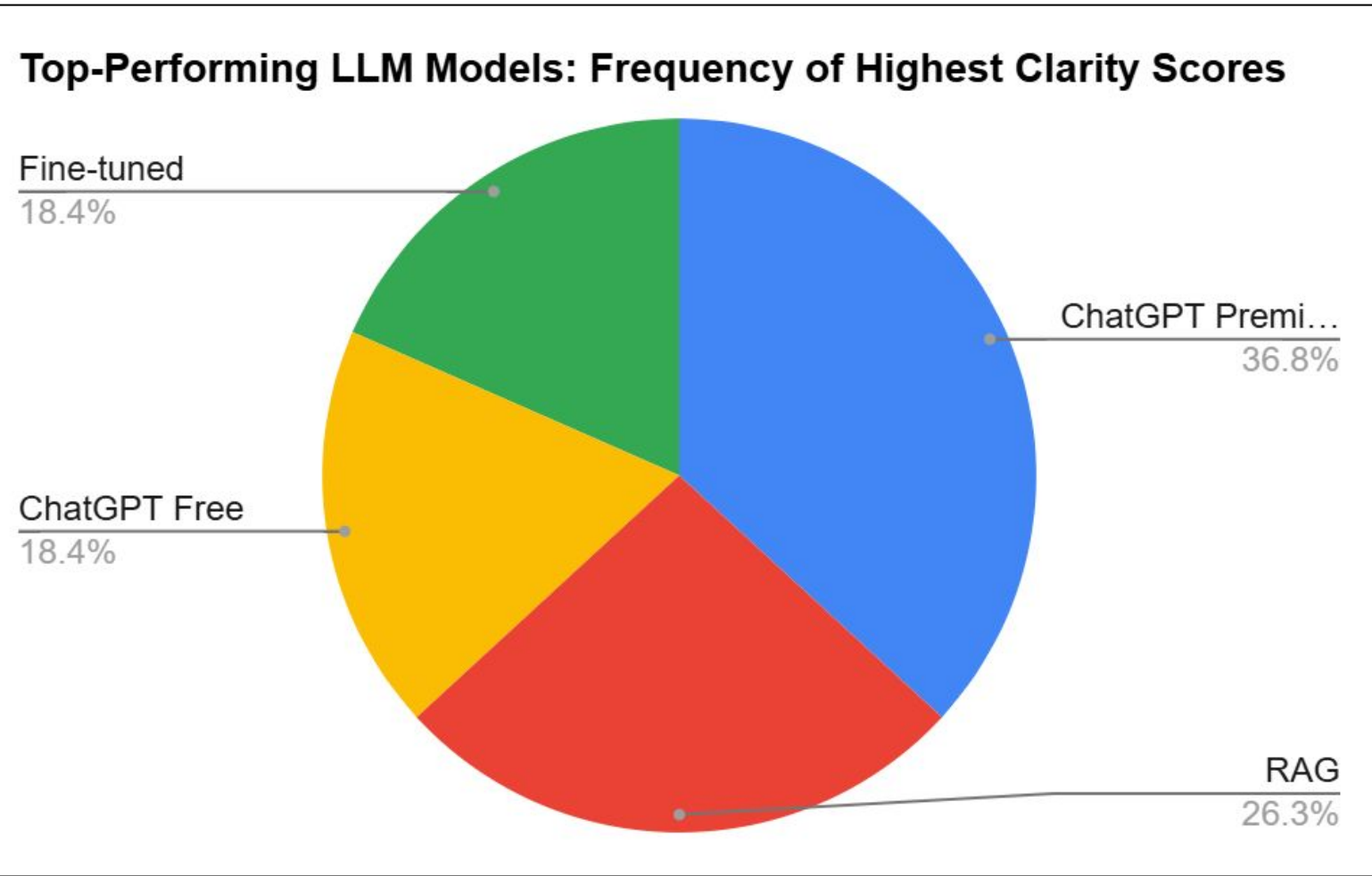
Results



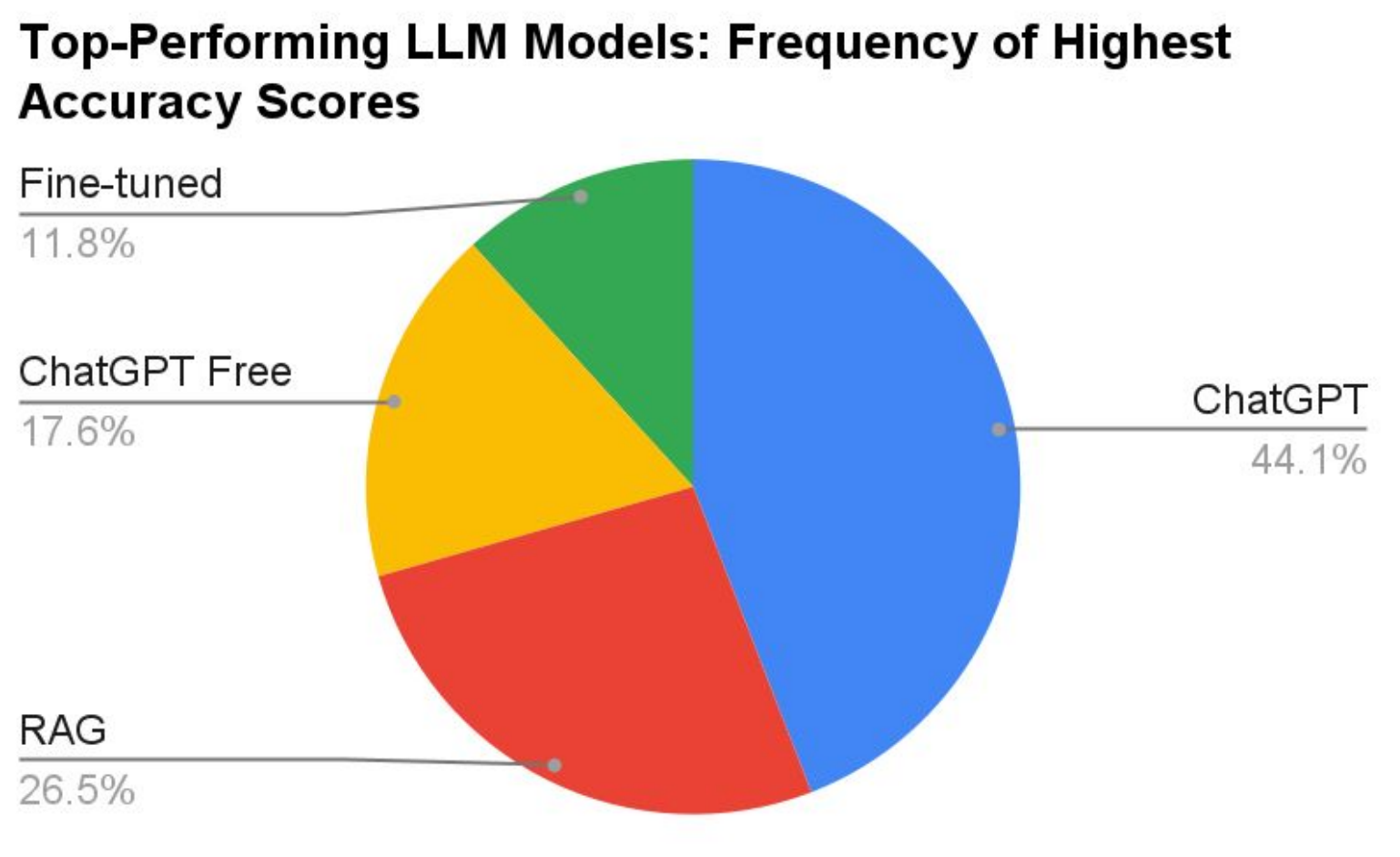
The scores for each category were averaged into a mean score with results showing the model that scored highest most often was ChatGPT Premium at 48.4% of the time. The model that had the least amount of high scores was the fine-tuned model with a frequency of 6.5%. The mean scores of each question and the performance of each model can be seen at the bottom of the section with the bar graph.



The pie chart titled "Top-Performing LLM Models: Frequency of Highest Relevance Scores" illustrates the comparative performance of different large language models based on their relevance scores. ChatGPT Premium leads the category with the highest frequency, accounting for 31.3% of the relevance scores. Following closely is the Fine-tuned model at 27.1%. The Retrieval-Augmented Generation (RAG) model holds a share of 22.9%, while ChatGPT Free comprises the smallest portion at 18.8%. This distribution suggests that ChatGPT Premium and Fine-tuned models are more effective in achieving high relevance scores compared to RAG and ChatGPT Free models.



Focusing on clarity, the model that scored the highest most often was again the ChatGPT Premium model with a frequency of 36.8%. The RAG model scored the highest 26.3% of the time, and both the fine-tuned model and the ChatGPT Free model had high scores the least often with each accumulating high scores only 18.4% of the time. The graph eludes that ChatGPT Premium provides the most comprehensible responses and is most effective for conveying information to a user.



In the pie chart titled "Top-Performing LLM Models: Frequency of Highest Accuracy Scores", the distribution of accuracy scores among the various large language models is demonstrated. ChatGPT Premium dominates with the highest frequency, capturing 44.1% of the accuracy scores, indicating its superior performance in this metric. The Retrieval-Augmented Generation (RAG) model follows with 26.5%, while ChatGPT Free accounts for 17.6%. The Fine-tuned model has the lowest frequency of high accuracy scores at 11.8%. This distribution highlights ChatGPT's effectiveness in achieving accuracy, with RAG also performing notably well compared to the other models.

Future Work

Future work includes further development of datasets in order to increase the accuracy of the RAG model. Collection and storing data on other courses from various disciplines would allow for users to obtain a more in-depth understanding of their options.

In addition to strengthening the functionality of the question-answering model, the front-end development of a chatbot can be implemented to allow for more user-friendly engagement. Currently, use is performed through a virtual IDE, Google Colab, and requires many execution steps. A user interface is essential for use by those who would benefit from the question-answering application.

To simplify the entire academic advising and course scheduling process, future work may include incorporating an Agent AI system that handles researching courses relevant to the users' major, selecting the most appropriate ones to take next, and registering the user [1]. This system would have to access a large database of course information and have access to the user's student account, but would ease the process of registering for courses and lessen the load of students requesting assistance from the university administration.

References

- Ibm, "What are ai agents?," IBM, <https://www.ibm.com/think/topics/ai-agents> (accessed Apr. 19, 2025).
- "Fine-tuning for retrieval augmented generation (RAG) with qdrant," OpenAI Cookbook, https://cookbook.openai.com/examples/fine-tuned_qdrant_retrieval_augmented_generation_qdrant (accessed Apr. 19, 2025).
- P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," arXiv.org, <https://arxiv.org/abs/2005.11401> (accessed Apr. 19, 2025).
- OpenAI API reference, <https://platform.openai.com/docs/api-reference/introduction> (accessed Apr. 18, 2025).
- "Build a retrieval augmented generation (RAG) app: Part 1," LangChain, <https://python.langchain.com/docs/tutorials/rag/> (accessed Apr. 19, 2025).
- "How to load pdfs," LangChain, https://python.langchain.com/docs/how_to/document_loader_pdf/ (accessed Apr. 19, 2025).
- LTU MCS Course Descriptions: <https://ltu.edu/academics/course-catalog/> (Accessed 4-20-25)